

# MIG: Momentum Integrated Gradients

## Transfer Attack on Face Verification

Abhishek Choudhary (MSIT)

July 25, 2026

# Selecting MIG

## Why MIG?

- Recent ICCV 2023 publication
- Official implementation available in TransferAttack repository
- Present in TransferAttack framework
- **Not implemented in intern baseline**

## Code

```
https://github.com/Abhichy18/transferattackInterns.git
```

# Understanding MIG

## MIG = Momentum Integrated Gradients targets Transferable Adversarial Attacks

### Integrated Gradients (IG)

- Approximates path integral across baseline to input
- Averages gradients over  $S=20$  interpolated steps
- Attributes key decision features for transferability

$$i\_grad = (adv - x\_base) * grad / S$$

### Momentum Update

- Like MI-FGSM, maintains accumulated gradient
- $g = decay * g + grad\_norm$
- $adv = adv + alpha * sign(g)$

### Path Interpolation

- Generates  $S$  scaled brightness images
- Prevents overfitting to surrogate model
- Improves transferability across models

### Attack Objectives (unchanged)

- Impersonation: maximize cosine similarity
- Dodging: minimize cosine similarity

# Adapting MIG to Face Verification

## Problem

- Official MIG tested on classification networks
- Baseline uses CNN-based DeepFace models:
  - ArcFace
  - Facenet512
  - GhostFaceNet
  - VGG-Face
- Direct copy-paste was impossible

## Adapted Core Ideas

- Batch-level linear path interpolation
- Embedding cosine similarity loss mapping
- Integrated gradient normalization
- Momentum accumulation
- Epsilon-ball projection
- Valid pixel range clipping  $[-1.0, 1.0]$

# Code Integration

## Step 1 — Register MIG

```
ALL_ATTACKS = [  
    'PGD', 'MI_FGSM',  
    'TI_FGSM', 'SI_NI_FGSM',  
    'MI_ADMIX_DI_TI',  
    'MIG' # <-- added  
]
```

## Step 3 — Dispatcher

```
if attack_name == "MIG":  
    return mig_attack(...)
```

## Step 2 — mig\_attack() Function

- 1 Initialize adversarial image
- 2 Create S interpolated image copies
- 3 Batch forward pass through surrogate model
- 4 Compute embedding similarity & loss
- 5 Compute gradient via GradientTape
- 6 Calculate integrated gradient (i\_grad)
- 7 Normalize integrated gradient
- 8 Update momentum
- 9 Project into epsilon-ball
- 10 Clip image to [-1.0, 1.0]
- 11 Repeat for NUM\_ITER

# Key Challenges Encountered

## Missing Evaluation Pipeline

- No script existed to compute similarity scores
- Only the summarization script was present
- Evaluation pipeline had been intentionally removed
- Had to reverse-engineer from baseline CSV structure

## Model Input Size Mismatch

- ArcFace, GhostFaceNet: 112 x 112
- Facenet512: 160 x 160
- VGG-Face: 224 x 224
- Fixed per-model resizing

## Dataset Path Mismatch

- Professor's CSV used /content/face\_module/...
- My Colab used dataset/...
- Solution: path conversion function

# Reconstructed Evaluation Pipeline

## Pipeline Flow

- 1 Load adversarial image
- 2 Load victim model
- 3 Compute embedding
- 4 Load target image
- 5 Compute target embedding
- 6 Compute cosine similarity
- 7 Store CSV row

## CSV Schema

```
row_id
attacker_model
img1
img2
dataset
attack_type
victim_model
attack_method
variant
similarity
```

## Output Files

- MIG\_raw\_similarities\_long.csv
- Merged into combined\_raw\_similarities\_long.csv
- Processed by build\_subset\_baselines.py

# Final Results

## MIG outperformed:

- SI-NI-FGSM (+2.71%)
- MI-FGSM (+8.13%)
- PGD (+17.71%)

## Why higher performance?

- Integrated gradients across 20 path steps
- Prevents overfitting to surrogate features
- Captures salient face representation paths
- Highly effective on CNN backbones

| Attack                | Breach Rate   |
|-----------------------|---------------|
| SI-NI-FGSM            | 31.46%        |
| <b>MIG (Proposed)</b> | <b>34.17%</b> |
| MI-FGSM               | 26.04%        |
| MI-ADMIX-DI-TI        | 23.96%        |
| TI-FGSM               | 20.00%        |
| PGD                   | 16.46%        |



# What I Learned

## Technical

- Transfer attack mechanics
- Face verification pipelines
- Reading and adapting research code
- Reverse engineering incomplete repositories
- Building missing evaluation infrastructure

## Process

- Integrating new research into existing codebases
- Quantitative attack comparison
- Debugging silent failures (missing rows, path mismatches)
- Extending baselines without redesigning the system

# Thank You

Abhishek Choudhary — MSIT

<https://github.com/Abhichy18/transferattacktInterns.git>